

1、总体方案

矩阵数据空间应以总线带宽对应的字节数对齐后偏移，提升搬运效率和缓存命中率。

设大矩阵大小为 $MSize$ ，线程数为 16、切分的小矩阵大小为 $BSize$ ，那么有 $MSize/BSize=N*N$ 个小矩阵。将大矩阵拆分成 $N*N$ 个小矩阵进行矩阵乘加运算，可以有效提升缓存命中率（小矩阵内存大小应该 $\leq L1$ 缓存大小）。同时可对小矩阵乘法采用 Winograd 算法。

假设要实现矩阵乘加操作为 $A*B+C=D$ ，遍历 A 和 B 中的每个小矩阵进行小矩阵乘运算（即 $N*N*N$ 次小矩阵乘运算）。取 A 矩阵第一行 N 个小矩阵乘数据放入 $L3Cache$ （前一行小矩阵数据地址连续，搬运时间短）， B 矩阵前 16 个小矩阵均分给 16 个线程做矩阵乘运算。以 $N=8$ 为例， A 矩阵的第一行和 B 矩阵的前两列小矩阵进行第一轮运算中，每个线程将实现对应小矩阵的矩阵乘操作。在最早完成矩阵乘操作的线程搬运 C 对应小矩阵进行矩阵加法。其他线程计算完成后直接搬入下 16 个线程需要的数据进行运算。

2、优化方法

2.1、大矩阵拆分成小矩阵提高缓存命中率

设大矩阵大小为 $MSize$ ，线程数为 16、切分的小矩阵大小为 $BSize$ ，那么有 $MSize/Bsize=N*N$ 个小矩阵。将大矩阵拆分成 $N*N$ 个小矩阵进行矩阵乘加运算，可以有效提升缓存命中率（小矩阵内存大小应该 $\leq L1$ 缓存大小）。

2.2、小矩阵乘法算法优化

在计算机中，乘法一般比加法慢，通过减少乘法次数，增加少量加法，可以实现加速。因此采用 Winograd 算法加速小矩阵乘运算。

2.3、线程对应数据搬运优化

假设要实现矩阵乘加操作为 $A*B+C=D$ ，遍历 A 和 B 中的每个小矩阵进行小矩阵乘运算（即 $N*N*N$ 次小矩阵乘运算）。取 A 矩阵第一行 N 个小矩阵乘数据放入 $L3Cache$ （前一行小矩阵数据地址连续，搬运时间短）， B 矩阵前 16 个小矩阵均分给 16 个线程做矩阵乘运算。以 $N=8$ 为例， A 矩阵的第一行和 B

矩阵的前两列小矩阵进行第一轮运算中，每个线程将实现对应小矩阵的矩阵乘操作。

2.4、数据计算搬运 pipeline

在最早完成矩阵乘操作的线程搬运 C 对应小矩阵进行矩阵加法。其他线程计算完成后直接搬入下 16 个线程需要的数据进行运算。